

OS ORANGE LEVEL

NAME: K RISHITH

SECTION: D

SRN: PEES1UG24CS207

PHASE 1:

1A) OUTPUT:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

1B) DIRECTORY STRUCTURE

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

PHASE 2:

2A) OUTPUT:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

2B) RAW BINARY FORMAT:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ^C
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09
227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                ES-VCS!.
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

PHASE 3:

3A) TESTING:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ echo "hello" > file1.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ echo "world" > file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes add file1.txt file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776789283 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776789293 6 file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

3B) FORMAT:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ echo "hello" > file1.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ echo "world" > file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes add file1.txt file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776789283 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776789293 6 file2.txt
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

PHASE 4:

4A) LOG:

```
./pes log
Initialized empty PES repository in .pes/
Committed: da69f851e01e... Initial commit
Committed: 9b8c9ffad06c... Add world
Committed: f395a539ad12... Add farewell
commit f395a539ad12fd99a8076215e7da053ef49d7aadcf72742fb62721916b98a212
Author: K Rishith <PES1UG24CS207>
Date: 1776790951

    Add farewell

commit 9b8c9ffad06c66a89b12a5d61091c9095e3921a19d2e7d0a3ea53dd75db5c087
Author: K Rishith <PES1UG24CS207>
Date: 1776790951

    Add worldell

commit da69f851e01e7b5ee08ae1d40e8fa892cb079fdc4595ba9fecdd5053b4e646eb6
Author: K Rishith <PES1UG24CS207>
Date: 1776790951

    Initial committial comQ

rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

4B) OBJECT STORE:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/9b/8c9ffad06c66a89b12a5d61091c9095e3921a19d2e7d0a3ea53dd75db5c087
.pes/objects/da/69f851e01e7b5ee08ae1d40e8fa892cb079fdc4595ba9fecdd5053b4e646eb6
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f3/95a539ad12fd99a8076215e7da053ef49d7aadcf72742fb62721916b98a212
.pes/refs/heads/main
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

4C) REFERENCE CHAIN:

```
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ cat .pes/refs/heads/main
f395a539ad12fd99a8076215e7da053ef49d7aadcf72742fb62721916b98a212
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
rishith@rishith-VirtualBox:~/PES1UG24CS207-pes-vcs$
```

PHASE 5:

Q5.1: Implementing pes checkout <branch>

- **File Changes:** You must update .pes/HEAD to point to the new branch reference (e.g., ref: refs/heads/<branch>).
- **Working Directory:** You must delete files that exist in the current branch but not the target, and overwrite files with the versions stored in the target branch's tree.
- **Complexity:** The "Update" phase is the most dangerous. If the process is interrupted, you could leave the working directory in a "half-switched" state. Furthermore, you must handle the index; the index must be updated to match the target branch's tree so that pes status shows no changes immediately after a checkout.

Q5.2: Detecting a "Dirty Working Directory" To detect a conflict, you perform a three-way comparison:

1. **Index vs. Working Directory:** Compare the mtime and size in the index to the actual file on disk. If they differ, the file is "dirty" (modified but uncommitted).
2. **Index vs. Target Branch Tree:** Compare the hash of the file in the index with the hash of the same file in the target branch's tree.
3. **The Rule:** If a file is dirty **AND** the version in the target branch is different from the version in your current branch, the checkout must refuse to proceed to prevent overwriting the user's uncommitted work.

Q5.3: The "Detached HEAD" State

- **What happens:** Commits still work; each new commit points to the previous one as its parent, and HEAD is updated with the new hash. However, because no branch file (like refs/heads/main) is being updated, there is no "name" attached to this line of history.
- **Recovery:** If the user switches back to main, they "lose" the hash of the last commit they made in the detached state. To recover, they must find the hash (via reflog in real Git or by searching the .pes/objects folder for the most recent commit object) and manually create a branch pointing to it: echo <hash> > .pes/refs/heads/new-branch.

PHASE 6:

Q6.1: The Mark-and-Sweep Algorithm

- **Algorithm:** 1. **Mark Phase:** Start at all "Roots" (all files in refs/heads/ and the HEAD file). Recursively traverse the objects: for each commit, find its tree and parent; for each tree, find its sub-trees and blobs. 2. **Sweep Phase:** Iterate through every file in .pes/objects. If a file's hash was not "marked" during the traversal, delete it.

- **Data Structure:** A **Bloom Filter** or a simple **Hash Set** (in-memory) is used to track reachable hashes efficiently.
- **Estimation:** For 50 branches, you visit 50 starting points. You visit all 100,000 commits. Since each commit has one root tree, and each tree has multiple blobs, you would likely visit **300,000 to 1,000,000+ objects** depending on the project's file count.

Q6.2: Concurrent GC Race Conditions

- **The Danger:** If GC runs while you are committing, it might see a "new" blob that you just wrote to the object store but haven't yet linked into a commit. Since no branch points to it yet, the GC marks it as "unreachable" and deletes it.
- **The Race Condition:**
 1. User runs `pes add file.txt`. A new blob `a1b2...` is written.
 2. GC starts, scans all branches, and sees that nothing points to `a1b2....`
 3. User runs `pes commit`. The code tries to reference `a1b2....`
 4. GC deletes `a1b2...` just as the commit is finishing. The repository is now corrupted (a "dangling" reference).
- **Git's Solution:** Git uses a **grace period** (usually 2 weeks). GC will only delete unreachable objects that are older than a specific timestamp (determined by the file's mtime). This ensures that "in-progress" objects are never deleted.